

Học máy và lập kế hoạch cho Orbit Wars: ctx2 – re-ranker học có giám sát, và NeRL – bộ lập kế hoạch dựa trên mô hình dự báo

Vũ Nguyên Đan, Phạm Tiến Dũng, Lê Hồng Anh

Nhóm 12 — Đội IAI-RL-DirtyDozen

MSSV: 23020351, 23020345, 23020327

Thành viên đại diện nộp bài: Vũ Nguyên Đan (23020351)

Môn học: Học tăng cường và lập kế hoạch

Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội (UET - VNU)

Hà Nội, Việt Nam

Tóm tắt nội dung—Báo cáo mô tả hai thiết kế agent khác nhau cho trò chơi chiến lược thời gian thực *Orbit Wars*, cả hai đều do nhóm xây dựng. Agent thứ nhất, *ctx2*, là một agent hybrid: một engine quy tắc xử lý toàn bộ vật lý và cơ chế trò chơi (sinh ứng viên, định cỡ hạm đội, dẫn đường quỹ đạo, tránh mặt trời, giải quyết giao tranh, gia cố phòng thủ), còn quyết định chiến lược quan trọng nhất—*chọn hành tinh nào để tấn công và từ đâu*—do một mạng nơ-ron nhỏ đóng vai trò *re-ranker* đảm nhiệm. Mạng đọc 230 đặc trưng cho mỗi ứng viên (46 đặc trưng cơ sở cộng thống kê tập kiểu DeepSet) và dùng MLP cỡ 230 → 128 → 128 → 1. Nó được huấn luyện bằng hồi quy có giám sát trên khoảng 510 000 ứng viên có nhãn, rồi xuất sang Python thuần để chạy trong giới hạn một giây mỗi lượt. Trên tập kiểm tra giữ lại, nó chọn đúng ứng viên hàng đầu như nhãn khoảng 81–85% số lần; điểm công khai khoảng 1001 (bản hiện tại 1000.8; từng gần 1067).

Agent thứ hai, NeRL (viết tắt của *Not even RL* – “thậm chí không phải RL”), là một bộ lập kế hoạch dựa trên mô hình dự báo. Nó *không* có tham số học. Nó dựng một mô hình mô phỏng tiến của trò chơi bằng tensor, dự báo chủ sở hữu và lực lượng đồn trú (số tàu đóng tại đó) tương lai của từng hành tinh trong một horizon thời gian, và chọn các đợt phóng theo mức độ chúng cải thiện trạng thái dự báo (một hàm mục tiêu dòng tàu cạnh tranh). Sau đó nó chọn vài đợt tấn công kiểu tham lam, gửi đúng hạm đội tối thiểu để chiếm (ngưỡng chiếm tối thiểu), gộp hạm đội từ nhiều hành tinh khi một hành tinh không đủ (focus-fire), và tái tập hợp tàu để phòng thủ. Trong đối đầu trực tiếp, nó thắng dứt khoát cả *ctx2* lẫn heuristic mạnh nhất của nhóm. Là bản nộp đang hoạt động trên bảng xếp hạng, nó đạt 1141.8, cao hơn re-ranker. Chúng tôi giải thích vì sao một bộ lập kế hoạch có mô hình chính xác tránh được trần bắt chước (*imitation ceiling*) và *distribution shift*—hai vấn đề vốn giới hạn re-ranker học có giám sát—và điều đó có ý nghĩa gì cho việc kết hợp học máy với lập kế hoạch.

Index Terms—Orbit Wars, lập kế hoạch dựa trên mô hình dự báo, horizon lùi, học có giám sát, re-ranker, DeepSet, game AI, self-play

I. Nhóm, đóng góp và kho mã nguồn

A. Nhóm và thành viên

Báo cáo do **Nhóm 12** theo danh sách lớp nộp, dưới tên đội IAI-RL-DirtyDozen. Thành viên và mã số sinh viên:

- Vũ Nguyên Đan (23020351) — thành viên đại diện nộp bài

- Phạm Tiến Dũng (23020345)
- Lê Hồng Anh (23020327)

B. Phân chia công việc và đóng góp

Trong tuần đầu (25/05–31/05), mỗi thành viên làm trên một nhánh riêng và thử một hướng khác nhau. Từ 01/06 đến 09/06, nhóm tập trung vào hướng mạnh nhất: lấy phần việc của thành viên có điểm bảng xếp hạng cao nhất làm nền, rồi phát triển tiếp, điều chỉnh theo điểm số. Bảng I liệt kê vai trò và đóng góp.

Bảng I
Vai trò và đóng góp của thành viên

Thành viên	Vai trò và đóng góp	Tỷ lệ
Vũ Nguyên Đan	Trưởng nhóm; agent <i>ctx2</i> ; viết báo cáo	42.5%
Phạm Tiến Dũng	Agent <i>NeRL</i> ; viết báo cáo	42.5%
Lê Hồng Anh	Hoạt động trong tuần đầu; tuần cuối không hoạt động	15%

C. Kho mã nguồn

Toàn bộ mã nguồn—cả hai agent, pipeline huấn luyện, khung đánh giá và mã nguồn báo cáo—đặt tại:

<https://github.com/dxpawn/OrbitWars>

D. Vị trí trên bảng xếp hạng (tóm tắt)

Tại thời điểm viết, IAI-RL-DirtyDozen có điểm công khai **1141.8**, hạng **636 / 4106** đội toàn cục, và đứng **thứ 12 trên 19** đội IAI-RL-* của lớp. Bảng đầy đủ trong Mục VI-C, Bảng V.

II. Giới thiệu

Orbit Wars là một bài toán chiến lược thời gian thực trên mặt phẳng: các hành tinh quay quanh một mặt trời ở trung tâm, mỗi hành tinh sinh tài nguyên và giữ tàu, người chơi gửi hạm đội đi chiếm hành tinh. Mỗi lượt, agent quyết định gửi bao nhiêu tàu, từ hành tinh nào, tới mục tiêu nào, cân bằng

giữa mở rộng, phòng thủ và tấn công. Không gian hành động lớn và chuyển động quỹ đạo phức tạp, nên tìm kiếm toàn cục đầy đủ trong giới hạn thời gian mỗi lượt là không khả thi.

Nhóm giải bài toán theo hai cách, và báo cáo trình bày cả hai như một phép so sánh.

Hướng A — học ở chỗ quan trọng (ctx2). Chúng tôi tách cơ học khỏi phán đoán. Mọi thứ có thể viết thành quy tắc—định cỡ hạm đội, ngắm bắn, tránh mặt trời, giao tranh, gia cố—do engine quy tắc xử lý. Chỉ riêng quyết định có giá trị cao, “trong các mục tiêu khả thi thì mục tiêu nào đáng tấn công”, là được học từ dữ liệu thu thập. Đây là một nguyên tắc đơn giản: không dùng học máy ở chỗ quy tắc đã chạy tốt; chỉ dồn mô hình vào quyết định đáng giá nhất.

Hướng B — lập kế hoạch với mô hình chính xác. Thay vì học một sở thích mục tiêu, bộ lập kế hoạch tính ra nó. Vì luật của trò chơi đã biết, ta mô phỏng tương lai gần một cách chính xác và chọn các đợt phóng làm kết quả dự báo có lợi cho ta nhất. Đây là lập kế hoạch dựa trên mô hình theo nghĩa horizon lùi / điều khiển dự báo theo mô hình (MPC) [4], [5]: mô hình được cho trước, không phải học.

Đóng góp.

- 1) Một agent hybrid engine + re-ranker (ctx2) với cách mã hóa tập bất biến hoán vị kiểu DeepSet, huấn luyện bằng học có giám sát ngoại tuyến và xuất sang Python không phụ thuộc thư viện (Mục IV).
- 2) Một bộ lập kế hoạch dựa trên mô hình dự báo dùng tensor: dự báo tiến quyền sở hữu lực lượng đồn trú và chấm điểm các đợt phóng theo ảnh hưởng biên của chúng lên một hàm mục tiêu đồng tàu cạnh tranh, kèm ngưỡng chiếm tối thiểu, focus-fire và tái tập hợp (Mục V).
- 3) Một so sánh trung thực giữa hai hướng, bàn về trần bất chức, distribution shift, và giá trị hạn chế của self-play cục bộ như một tín hiệu xếp hạng (Mục VII).

III. Bài toán và cơ chế trò chơi

Bàn chơi có kích thước 100×100 với mặt trời bán kính $R_{\text{sun}} = 10$ ở trung tâm. Hành tinh có thể đứng yên hoặc chuyển động trên quỹ đạo; tốc độ hạm đội tối đa là 6 đơn vị mỗi lượt và một trận kéo dài tối đa 500 lượt. Mỗi lượt, agent nhận một *observation* gồm mọi hành tinh (chủ sở hữu, số tàu, sản lượng, vị trí), mọi hạm đội đang bay, và các đe dọa sắp tới. Tốc độ hạm đội phụ thuộc số tàu; một hạm đội nhắm hành tinh đang di chuyển phải ngắm đón trước; và một hạm đội bị tiêu diệt nếu đường bay cắt qua mặt trời. Để chiếm một hành tinh, hạm đội phải đến nơi với số tàu nhiều hơn số tàu phòng thủ tại thời điểm đến; sau đó sản lượng của hành tinh thuộc về chủ mới.

Các luật này hoàn toàn biết trước và xác định một khi đã cho hành động của người chơi. Đây chính là điều làm Hướng B khả thi: một hàm chuyển trạng thái đã biết có thể được mô phỏng thay vì học.

IV. Hướng A: engine quy tắc và re-ranker học (ctx2)

A. Phạm vi của thành phần học

ctx2 **không** học một chính sách đầy đủ. Mạng chỉ dùng ở bước *re-ranking*: sau khi engine liệt kê các ứng viên tấn công khả thi cho mỗi hành tinh nguồn, mạng cho mỗi ứng viên một điểm chất lượng và điều chỉnh thứ tự ưu tiên. Mọi thứ còn lại—số tàu cần để thắng và giữ, ngắm đón mục tiêu di chuyển, tránh bóng mặt trời, xử lý counter-snipe, gia cố hành tinh bị đe dọa—do engine quy tắc trong engine/orbit_base.py thực hiện.

B. Pipeline quyết định mỗi lượt

Pipeline có sáu bước:

- 1) **Đọc observation.** Giải mã trạng thái thành các hành tinh (chủ sở hữu, số tàu, sản lượng, vị trí quỹ đạo), hạm đội đang bay, tổng lực lượng theo phe, và các đe dọa sắp tới.
- 2) **Sinh ứng viên.** Với mỗi hành tinh ta sở hữu, engine liệt kê các mục tiêu nó có thể tới, sắp xếp sơ bộ theo heuristic khoảng cách có trọng số. Vị trí của mỗi ứng viên trong danh sách gộp được ghi là chỉ số idx . Hệ số mở rộng $\text{expand} = 10$ giữ thêm ứng viên trước khi cắt về top K .
- 3) **Tạo đặc trưng.** Mỗi cặp (nguồn, đích) được chuyển thành một vector 46 chiều (Mục IV-E).
- 4) **Chấm điểm bằng mạng.** `score_many(rows)` nhận toàn bộ ứng viên của một lần gọi, thêm thống kê tập (mean, max, min, std theo từng chiều), chuẩn hóa, rồi chạy MLP.
- 5) **Re-rank.** Ưu tiên cuối cùng là

$$\text{priority} = \text{idx} - \text{bonus} \cdot \sigma(\text{score}), \quad (1)$$

với σ là sigmoid và $\text{bonus} = 1,45$ (2p) hoặc $1,25$ (4p). Điểm học cao kéo ứng viên lên so với thứ tự heuristic; bonus giới hạn mức mạng có thể thay đổi thứ tự. Ở 4p, một jitter nhỏ (10^{-6}) phá hòa.

- 6) **Thực thi.** Engine gửi hạm đội tới các mục tiêu đã chọn: tính số tàu cần để thắng và giữ, ngắm đón mục tiêu di chuyển, tránh mặt trời, và chạy một vòng gia cố phòng thủ.

Suy luận mạng khoảng **0,1 giây/lượt**, thoải mái trong giới hạn một giây.

C. Engine quy tắc

Engine trong orbit_base.py chứa phần lớn logic trò chơi dạng quy tắc:

- **Định cỡ và dẫn đường hạm đội:** tốc độ theo số tàu, dự báo vị trí mục tiêu lúc đến (ngắm đón), kiểm tra và chạm mặt trời.
- **Heuristic chọn mục tiêu ban đầu:** khoảng cách có trọng số, ưu tiên hành tinh đứng yên, một bonus mở rộng, và xử lý trường hợp nhiều phe cùng nhắm một mục tiêu.
- **Mô phỏng tiến ngắn (4p):** một look-ahead 7 lượt để chỉnh thứ hạng ứng viên cho giá trị xa hơn một bước.

- **Phòng thủ chủ động:** phát hiện snipe và counter-snipe, gia cố hành tinh yếu, tái chiếm muôn ở 2p.
- **Điều khiển đa hành động:** giới hạn số hành động mỗi lượt (7 ở 2p, 5 ở 4p) và một tìm kiếm mở rộng theo nguồn.

Tách engine khỏi mạng giúp gỡ lỗi: cơ chế và phần sinh ứng viên có thể kiểm thử riêng, và re-ranker chỉ được bật khi cần đánh giá phân học.

D. Kiến trúc mạng

Re-ranker là một MLP ba lớp với kích hoạt ReLU, huấn luyện bằng hồi quy có giám sát để dự đoán điểm chất lượng mục tiêu từ vector đặc trưng. Sau huấn luyện, trọng số được xuất sang Python thuần—không cần PyTorch hay TensorFlow lúc thi đấu.

a) *Đầu vào 230 chiều và mã hóa DeepSet:* Mỗi ứng viên i trong một quyết định có vector cơ sở $\mathbf{x}_i \in \mathbb{R}^{46}$. Trước MLP, ta thêm thông kê tập trên toàn bộ ứng viên S của quyết định đó:

$$\boldsymbol{\mu} = \text{mean}_{j \in S}(\mathbf{x}_j), \quad \mathbf{m} = \max_{j \in S}(\mathbf{x}_j), \quad (2)$$

$$\mathbf{n} = \min_{j \in S}(\mathbf{x}_j), \quad \boldsymbol{\sigma} = \text{std}_{j \in S}(\mathbf{x}_j), \quad (3)$$

(mỗi giá trị tính theo từng chiều). Đầu vào đầy đủ là $\mathbf{z}_i = [\mathbf{x}_i; \boldsymbol{\mu}; \mathbf{m}; \mathbf{n}; \boldsymbol{\sigma}] \in \mathbb{R}^{230}$.

Đây là một **DeepSet-style set-pooling** [1]: mạng đánh giá một ứng viên so với các lựa chọn khác trong cùng quyết định, chứ không tách riêng, mà vẫn chấm điểm từng ứng viên một cách rõ ràng và riêng lẻ. Mô hình **bất biến hoán vị** theo thứ tự ứng viên và xử lý được số ứng viên thay đổi mỗi lượt.

b) *Forward pass:* Sau chuẩn hóa $\hat{\mathbf{z}} = (\mathbf{z} - \boldsymbol{\mu}_{\text{train}})/\boldsymbol{\sigma}_{\text{train}}$ (các hằng MEAN và STD lưu trong file trọng số):

$$h_1 = \text{ReLU}(W_0 \hat{\mathbf{z}} + b_0), \quad h_2 = \text{ReLU}(W_1 h_1 + b_1), \quad s = W_2 h_2 + b_2. \quad (4)$$

Kiến trúc là $230 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{linear}} 1$. Vô hướng s , sau một sigmoid, được dùng trong (1). Ta giữ hai bộ trọng số riêng (`student_weights_2p.py`, `student_weights_4p.py`) vì phân bố trạng thái và các thiết lập re-rank khác nhau giữa 2p và 4p.

E. Bộ 46 đặc trưng cơ sở

Các đặc trưng được tạo trong `engine/main.py` và kết hợp trạng thái toàn cục (dùng chung mọi ứng viên trong một lượt) với thông tin cục bộ của cặp (nguồn, đích). Hầu hết giá trị được *clamp* về $[0, 1]$ hoặc $[-1, 1]$; số tàu đi qua một hàm bão hòa $x/(x+40)$ để giảm ảnh hưởng của quy mô tuyệt đối. Bảng II liệt kê đủ 46 chiều theo thứ tự.

Một lớp lịch sử ngắn (`_FeatureHistory`) giữ buffer 5–10 lượt để tính *momentum*, *aggression_trend*, *enemy_rhythm*, *approach_rate*, và *convergence_threat*—các đặc trưng xu hướng giúp mạng cảm nhận giai đoạn trận và áp lực đối thủ thay vì chỉ một ảnh chụp. Các thiết lập re-rank nằm trong `engine/feature46_manifest.json`: `bonus_2p=1.45`, `bonus_4p=1.25`, `expand=10`.

F. Phương pháp huấn luyện

Mô hình được huấn luyện bằng **học có giám sát ngoại tuyến**: thu thập các cặp (đặc trưng $\rightarrow$ điểm chất lượng), rồi khớp chúng. Không có reward từ môi trường và không có exploration—chỉ là hồi quy MSE trên các nhãn cố định.

Nhãn chất lượng. Với mỗi quyết định, một pipeline chấm điểm nội bộ xếp hạng từng ứng viên từ 46 đặc trưng và ngưỡng cảnh tập của nó. Điểm này là mức ưu tiên chiến lược do bộ chấm điểm tham chiếu gán, và là mục tiêu hồi quy cho MLP nhỏ hơn.

Thu thập dữ liệu.

- Chạy agent qua khoảng **250 trò chơi** (2p và 4p) đối đầu một pool hỗn hợp (`heuristic_v6`, `adv_hellburner`, `adv_proto_v15`, `adv_lb958`).
- Mỗi lần gọi `score_many` được gán một *group id*; mọi ứng viên trong một lần gọi cùng một nhóm, nên ngưỡng cảnh tập có thể dựng lại lúc huấn luyện.
- Quy mô: khoảng 25 000 nhóm quyết định (2p), khoảng 37 000 (4p); tổng khoảng **510 000** ứng viên có nhãn.

Huấn luyện.

- Ghép 46 đặc trưng với mean/max/min/std của nhóm $\Rightarrow$ 230 chiều; chuẩn hóa theo mean và std toàn tập.
- Chia 90/10 *theo nhóm* (không theo dòng) để tránh rò rỉ giữa train và validation.
- Hàm mất MSE giữa điểm dự đoán và điểm mục tiêu; optimizer Adam trên GPU; huấn luyện khoảng **1 phút**.
- Metric chính: *held-out group top-1 agreement*—trong mỗi nhóm validation, ứng viên hàng đầu của mô hình có trùng nhãn không?

Xuất mô hình. Trọng số, MEAN, và STD được ghi vào `model/student_weights_{2p,4p}.py`; `score_many` dựng lại set-pooling và forward pass đúng như lúc huấn luyện.

V. Hướng B: NeRL, bộ lập kế hoạch dựa trên mô hình dự báo

NeRL—viết tắt của *Not even RL*, vì nó hoàn toàn không học—chơi cùng trò chơi với `ctx2` nhưng theo cách ngược lại: thay vì *học* mục tiêu nào nên ưu tiên, nó *mô phỏng* kết quả của từng đợt và chọn đợt cải thiện kết quả dự báo nhiều nhất. Cái tên cho thấy NeRL thuộc phần *lập kế hoạch* của môn học (lập kế hoạch dựa trên mô hình đã biết), không phải phần học máy. Nó **không có mạng nơ-ron và không có trọng số học**; `torch` chỉ được dùng như một thư viện mảng nhanh để toàn bộ mô phỏng vừa với ngân sách thời gian.

A. Agent này là gì, theo ngôn ngữ RL

Vì luật đã biết chính xác, NeRL là **lập kế hoạch dựa trên mô hình cho trước** [5]—dự báo theo mô hình (MPC) [4]. Nó *không* phải heuristic phản xạ (nó không chấm điểm trạng thái hiện tại; nó mô phỏng tương lai), và *không* học. Các phần heuristic duy nhất là những xấp xỉ thực dụng để giữ cho việc lập kế hoạch nhanh: rút gọn danh sách ứng viên, lựa chọn tham lam (không tối ưu), một horizon cố định, và giả định rằng đối thủ không phóng gì mới ngoài các hạm đội đã bay (không có tìm kiếm cây trò chơi đối kháng).

Bảng II
Bộ 46 đặc trưng cơ sở (thứ tự như trong _CANDIDATE_FEATURES)

STT	Tên	Mô tả ngắn
<i>Nhóm trạng thái toàn cục (chỉ số 0–11)</i>		
0	game_progress	Lượt hiện tại / 500
1	players_alive	Số người chơi còn hoạt động / 4
2	my_planet_share	Tỷ lệ hành tinh ta sở hữu
3	my_gdp_share	Tỷ lệ tổng sản lượng ta sở hữu
4	my_ship_share	Tỷ lệ tổng tàu (hành tinh + transit) ta sở hữu
5	momentum	Chênh lệch tỷ lệ tàu ta so với 5 lượt trước
6	my_fleet_ratio	Tỷ lệ tàu đang transit / tổng tàu ta
7	leader_gap	Khoảng cách lực lượng tới phe dẫn đầu (chuẩn hóa)
8	am_i_targeted	Tỷ lệ hạm đội đối thủ đang nhắm hành tinh ta
9	fastest_grower_gap	Chênh momentum giữa đối thủ tăng nhanh nhất và ta
10	aggression_trend	Xu hướng mức độ bị nhắm (5 lượt)
11	enemy_rhythm	Biến thiên nhịp tấn công địch (10 lượt)
<i>Nhóm hành tinh nguồn và ngoại giao (12–19)</i>		
12	src_ships	Số tàu nguồn (bão hòa)
13	src_production	Sản lượng nguồn / 5
14	src_safety	Mức an toàn nguồn trước đe dọa sắp tới
15	diplomacy	+1 phòng thủ ta, 0 trung lập, -1 tấn công địch
16	target_production	Sản lượng đích / 5
17	is_dynamic_target	1 nếu sao chổi hoặc hành tinh không tĩnh
18	owner_power	Tỷ lệ tàu của chủ đích so với toàn bản
19	is_weakest_enemy	1 nếu đích thuộc phe địch yếu nhất
<i>Nhóm mô phỏng chiến và đe dọa (20–31)</i>		
20	proj_owner	Dự báo chủ đích lúc đến: ta / trung lập / địch
21	proj_ships	Số tàu phòng thủ dự báo lúc đến (bão hòa)
22	threat_after	Đe dọa địch đến <i>sau</i> khi ta chiếm
23	support_after	Hỗ trợ ta đến sau khi ta chiếm
24	pre_volatility	Biến động lực lượng trước khi ta đến
25	threat_potential	Tiềm năng tấn công từ hành tinh địch khác
26	support_potential	Tiềm năng hỗ trợ từ hành tinh ta khác
27	eta	Thời gian đến / 25
28	send_fraction	Tỷ lệ tàu gửi / tổng tàu ta
29	need_ratio	Tàu cần để thắng / tàu nguồn
30	margin	Biên an toàn (send – need), chuẩn hóa
31	can_win	1 nếu gửi đủ tàu để chiếm
<i>Nhóm chiến lược và địa hình (32–45)</i>		
32	garrison_strength	Lực lượng còn lại sau chiếm (bão hòa)
33	defense_sustainability	Khả năng giữ sau chiếm
34	target_roi	Sản lượng đích / (tàu cần +1)
35	front_diff	Chênh khoảng cách tới địch gần nhất (nguồn vs. đích)
36	avg_enemy_distance	Khoảng cách trung bình tới trọng tâm địch
37	angular_spread	Mức độ địch bao quanh ta theo góc
38	orbital_trend	Xu hướng ETA do chuyển động quỹ đạo
39	convergence_threat	Mức đe dọa hội tụ (địch thu hẹp khoảng cách)
40	approach_rate	Tốc độ địch tiến tới trọng tâm ta
41	enemy_commitment	Tỷ lệ tàu địch đang transit
42	weakest_colony	Hành tinh ta yếu nhất (sau khi trừ đe dọa sắp tới)
43	local_superiority	Ưu thế cục bộ hỗ trợ so với đe dọa
44	economic_impact	Sản lượng đích / sản lượng ta
45	enemy_exposed	Mức độ sơ hở của phe địch

B. Pipeline mỗi lượt

Thuật toán trong run_turn:

- 1) **Đọc** observation thành tensor (hành tinh, hạm đội đang bay).
- 2) **Dừng hoặc làm mới mô hình chuyển động.** PlanetMovement dự báo vị trí mọi hành tinh trong horizon H , theo dõi hạm đội đang bay bằng một sổ cái (ledger), và hỗ trợ dự báo tiến lực lượng đồn trú.
- 3) **Cache khoảng cách.** Khoảng cách chéo thời gian “hành tinh s tại $t=0$ đến hành tinh t tại bước k ”—khoảng cách đúng về hình học để kiểm tra xem một hạm đội có kịp tới một hành tinh đang di chuyển hay không.
- 4) **Dự báo đồn trú.** garrison_status trả về, cho các bước $0 \dots H$, dự báo (chủ sở hữu, số tàu) của từng hành tinh bằng một đệ quy *chính xác*: sản lượng, hạm đội đến, và giao tranh được giải quyết theo từng bước.
- 5) **Lập kế hoạch các đợt sóng** (Mục V-C).
- 6) **Xuất** các đợt phóng đã chọn dưới dạng payload hành động thừa.

C. Lập kế hoạch đợt sóng và chấm điểm

a) *Danh sách rút gọn*: Nguồn là các hành tinh ta sở hữu có ít nhất min_ships_to_launch tàu, xếp theo số tàu. Mục tiêu gồm một danh sách *tấn công* (hành tinh địch hoặc trung lập theo khoảng cách) và một danh sách *phòng thủ* các *mục tiêu lật (flip)*—hành tinh của ta được dự báo sẽ mất trong horizon, xếp theo độ khẩn cấp = sản lượng · (số lượt còn lại) + số tàu.

b) *Định cỡ và khả năng tới được*: Với mỗi (nguồn, đích), cỡ đợt phóng là safe_drain: số tàu nhiều nhất nguồn có thể gửi mà vẫn tự giữ được qua horizon. Một kiểm tra khả năng tới được dạng giải tích (có tính đến trôi quỹ đạo, mặt trời che tầm nhìn, và độ lệch bề mặt) cùng một bộ giải intercept_angle (lập điểm bất động) cho ra góc ngắm và ETA tới mục tiêu di chuyển.

c) *Ngưỡng chiếm tối thiểu*: capture_floor tính số tàu tối thiểu cần để chiếm một mục tiêu tại lượt đến của nó:

$$f_k = \lceil \text{phòng thủ}_k + \text{phụ trội} + \text{tiếp viện}_k \rceil, \quad (5)$$

với phòng thủ _{k} là lực lượng đồn trú dự báo tại bước đến k . Một ứng viên chỉ được giữ nếu cỡ của nó vượt f_k . Điều này cho cách dùng tàu hiệu quả: agent không bao giờ gửi nhiều hơn mức cần cho một lần chiếm.

d) *Focus-fire*: Khi không nguồn đơn nào vượt được ngưỡng của một mục tiêu, bộ lập kế hoạch *gộp* nhiều nguồn có hạm đội đến trong *cùng* một lượt và tổng số tàu vượt f_k , tạo một đòn đánh phối hợp đa nguồn. Đây là tính năng chính mà cấu hình mạnh nhất bổ sung so với bản đơn nguồn.

e) *Chấm điểm cạnh tranh biên*: Mỗi tập đợt phóng ứng viên c được chấm theo ảnh hưởng của nó lên dòng tàu dự báo. Gọi $\Delta\eta_j(c)$ là thay đổi số tàu rỗng dự báo của người chơi j (sản xuất trừ mất trong giao tranh qua horizon) do c gây ra. Hàm mục tiêu là

$$s(c) = \Delta\eta_{\text{me}}(c) - \sum_{j \neq \text{me}} \Delta\eta_j(c). \quad (6)$$

Đây là một look-ahead một bước (one-ply) trên bộ mô phỏng chính xác: nó thường cho các đợt phóng làm tăng số tàu rỗng của ta và giảm số tàu rỗng của đối thủ.

f) *Lựa chọn tham lam và tái tập hợp*: Một vòng tham lam chọn các đợt sóng có điểm cao nhất không xung đột, tối đa max_waves_per_turn, trong ngân sách của từng nguồn và trên một ngưỡng ROI (chỉ phóng nếu $s(c) > \text{roi_threshold}$). Cuối cùng, một bước *tái tập hợp* chuyển tàu dư từ các hành tinh an toàn về các hành tinh bạn đang bị đe dọa, theo một gradient áp lực dịch có tính cả hạm đội địch đang bay.

D. Cấu hình

Một ProducerLiteConfig cố định giữ các thiết lập. Cấu hình 2p dùng horizon = 18, tối đa 6 đợt sóng mỗi lượt, ROI = 1,5, và focus-fire bật (tối đa 4 nguồn đánh). Cấu hình 4p rút horizon xuống 13 và giảm số nguồn, số mục tiêu phòng thủ, và số nguồn đánh, vì một trận bốn người có nhiều phân nhánh hơn và tầm look-ahead hữu ích ngắn hơn.

VI. Kết quả thực nghiệm

A. ctx2: độ khóp nhãn trên tập giữ lại

Bảng III
Metric huấn luyện ctx2 trên tập giữ lại (chia theo nhóm)

Metric	2p	4p
Group top-1 agreement	81,3%	84,8%
R^2 (MSE trên điểm)	0,917	0,933
Tổng quát: top-1 $\approx 85\%$, top-3 $\approx 95\%$, $R^2 \approx 0,93$		

Top-1 agreement khoảng 85% nghĩa là trong khoảng 85% quyết định re-rank, ứng viên hàng đầu của mô hình trùng với nhãn; top-3 khoảng 95% nghĩa là mục tiêu có nhãn cao nhất thường nằm trong top ba của mô hình, nên lỗi chủ yếu ở thứ tự phụ chứ không phải bỏ sót mục tiêu chính. Điểm công khai khoảng **1000.8** (từng gần 1067; điểm TrueSkill dao động khoảng ± 30). Suy luận khoảng 0,1 giây/lượt, và bản nộ chỉ cần Python chuẩn lúc thi đấu.

B. NeRL thắng các agent khác của nhóm trong đối đầu

Trong đối đầu trực tiếp (khung cục bộ của nhóm, dựng trên kaggle_environments), NeRL rõ ràng mạnh hơn cả heuristic lẫn ctx2:

Bảng IV
NeRL so với các agent khác của nhóm (2p, seed tiêu biểu)

Trận (NeRL = P0)	Thắng	Điểm	Lượt thắng
NeRL vs. heuristic_v6	NeRL	1211–0	85 / 500
NeRL vs. ctx2	NeRL	1478–0	94 / 500

Trong cả hai trận 2p, NeRL loại đối thủ trước lượt 100 trên 500. Là bản nộ đang hoạt động, nó đạt **1141.8**—cao hơn 1000.8 của ctx2 và nhất quán với kết quả đối đầu, nhưng chỉ ở giữa của toàn bộ 4106 đội (Mục VI-C). Hai con số đo hai

thứ khác nhau: kết quả đối đầu là một trận tay đôi với chính các agent của nhóm, còn điểm bảng xếp hạng là xếp hạng trước toàn bộ quần thể. Chi phí tính mỗi lượt (torch trên CPU) vẫn dưới giới hạn một giây dù chạy một dự báo tiến đầy đủ mỗi lượt.

C. Vị trí trên bảng xếp hạng

Bảng V cho thấy nhóm trong các đội IAI-RL-* của lớp trên bảng xếp hạng công khai (ảnh chụp 09/06; điểm TrueSkill dao động khoảng ± 30 và còn thay đổi khi các bản nộp tiếp tục thi đấu). Bản nộp đang hoạt động của nhóm là **NeRL** ở 1141.8; bản *ctx2* đạt 1000.8. Trong 4106 đội toàn cục, nhóm xếp hạng 636; trong 19 đội của lớp, nhóm xếp hạng 12. Chúng tôi nói thắng: NeRL thắng các agent *trước đó của chính nhóm* trong đối đầu, nhưng so với toàn bộ các đội thì nó ở giữa bảng, và vài đội của lớp dùng các agent công khai mạnh tương tự xếp trên nhóm. Vì vậy giá trị của dự án nằm ở phần so sánh vì sao lập kế hoạch thắng re-ranker học của nhóm (Mục VII), hơn là bản thân con số.

Bảng V
Các đội IAI-RL-* của lớp trên bảng xếp hạng công khai (ảnh chụp 09/06, tổng 4106 đội)

Hạng lớp	Đội	Điểm	Hạng toàn cục
1	IAI-RL-04	1294.9	79
2	IAI-RL-IX	1243.8	143
3	IAI-RL-TIm71	1235.6	158
4	IAI-RL-MT	1224.9	207
5	IAI-RL-codex_v1	1200.7	314
6	IAI-RL-NapNap	1197.5	341
7	IAI-RL-Beginner	1197.2	342
8	IAI-RL-codex	1187.9	399
9	IAI-RL-03	1186.4	408
10	IAI-RL-17	1183.3	430
11	IAI-RL-II	1155.3	591
12	IAI-RL-DirtyDozen (nhóm)	1141.8	636
13	IAI-RL-Checkm8	1141.4	638
14	IAI-RL-003	1120.4	697
15	IAI-RL-Cherry_11	1119.4	700
16	IAI-RL-HaiPhong	969.7	926
17	IAI-RL-06	923.0	1022
18	IAI-RL-10	803.6	1421
19	IAI-RL-Trauma07	709.0	1906

VII. Thảo luận: học máy và lập kế hoạch

A. Trần của học có giám sát ngoại tuyến

Mục tiêu của *ctx2* là *khớp nhân*, không phải *thắng nhiều trận hơn*. Thực nghiệm của chúng tôi cho thấy độ khớp nhân cao hơn không phải lúc nào cũng chơi mạnh hơn—một phiên bản khớp nhân sát hơn lại đạt điểm *thấp hơn*—nên **độ khớp nhân không bằng sức mạnh thi đấu**. Một re-ranker có giám sát, giỏi nhất, chỉ đạt tới chất lượng của bộ chấm điểm đã tạo ra nhân của nó; nó không thể vượt qua [3]. Đây là cái trần mà bộ lập kế hoạch tránh được: bằng cách tối ưu một *kết quả* (dòng tàu dự báo) thay vì bắt chước một *sở thích*, bộ lập kế hoạch không bị giới hạn bởi bất kỳ “thầy” nào.

B. Vì sao bộ lập kế hoạch thắng

Bộ lập kế hoạch có ba lợi thế cấu trúc so với re-ranker:

- 1) **Tối ưu kết quả, không phải nhân**. (6) dựa trên động lực thật của trò chơi; re-ranker dựa trên một heuristic chấm điểm cố định.
- 2) **Dùng tàu hiệu quả**. Ngưỡng chiếm gửi đúng hạm đội thắng nhỏ nhất, giải phóng tàu cho thêm mục tiêu—hiệu quả mà re-ranker dựa-đặc-trung chỉ xấp xỉ.
- 3) **Phối hợp**. Focus-fire chạy các đòn đa hành tinh mà một re-ranker chấm từng đợt phóng riêng lẻ không thể biểu diễn.

C. Distribution shift và lỗi lũy tích

Lúc huấn luyện, dữ liệu của *ctx2* nằm trên phân bố trạng thái do pipeline thu thập tạo ra; lúc thi đấu, agent đi theo phân bố *của chính nó*, và sự lệch này gây **lỗi lũy tích**. Cách sửa chuẩn là **Dagger** [2]: gán nhãn lại các trạng thái mà agent thực sự gặp rồi huấn luyện lại. Bộ lập kế hoạch không gặp vấn đề này—nó lập lại kế hoạch từ trạng thái thật mỗi lượt, nên không có khoảng cách phân bố train/test.

D. Độ tin cậy của đánh giá

Self-play cục bộ và win-rate trước một pool cố định *không* khớp chặt với bảng xếp hạng công khai, do distribution shift sang quần thể đối thủ thật và drift TrueSkill. Vì vậy chúng tôi chỉ tin **metric cặp đối trên tập giữ lại** (agreement, R^2) và **kết quả trực tiếp trên bảng xếp hạng**, và thận trọng với mọi so sánh điểm tuyệt đối giữa các ngày.

E. Giới hạn và hướng phát triển

Đơn giản hóa lớn nhất của bộ lập kế hoạch là nó không mô hình hóa đối thủ: nó giả định đối thủ không phóng gì mới ngoài các hạm đội đã bay. Các mở rộng hữu ích hướng tới mô hình hóa đối thủ gồm (i) một roll-out đối kháng nông (mô phỏng nước đi tốt nhất khả dĩ của đối thủ mạnh nhất), (ii) thay lựa chọn tham lam bằng một beam search nhỏ, và (iii) dùng điểm học của *ctx2* để phá hòa trong danh sách rút gọn của bộ lập kế hoạch—một cách cụ thể để kết hợp hai hướng, với học máy đề xuất ứng viên và lập kế hoạch đưa ra lựa chọn cuối.

VIII. Cấu trúc triển khai

Bảng VI cho thấy gói mô hình đóng gói; toàn bộ kho phát triển—cả hai agent, pipeline huấn luyện, và khung đánh giá—đặt tại liên kết GitHub trong Mục I.

Tải tạo (Hướng A): thu thập → huấn luyện → validate → build submission; dữ liệu .npz (≈ 85 MB) có thể dựng lại bằng script thu thập. Hướng B không có bước huấn luyện—agent tự chứa và xác định với một seed cho trước.

IX. Kết luận

Chúng tôi trình bày hai agent cho Orbit Wars và một so sánh trung thực giữa hai hướng. *ctx2* là một thiết kế hybrid—engine quy tắc cho cơ học và một re-ranker DeepSet nhỏ (230-128-128-1) cho việc chọn mục tiêu—huấn luyện trên khoảng 510 000 ứng viên, đạt khoảng 81–85% top-1 agreement và $R^2 \approx 0,92$ – $0,93$, với điểm khoảng 1000.8. NeRL, bộ lập kế

Bảng VI
Bố cục bản nộp đóng gói (gói mô hình ctx2); NeRL nộp dưới dạng một file
độc lập

Đường dẫn	Vai trò
<i>Hướng A — ctx2 (re-ranker học)</i>	
agent/submission_distilled_ctx2.py	Bản nộp thi đấu (một file)
model/student_weights_{2p,4p}.py	Trọng số MLP + score_many
engine/orbit_base.py	Engine quy tắc (gameplay)
engine/main.py	Sinh ứng viên, 46 đặc trưng, re-rank
engine/feature46_manifest.json	Schema đặc trưng + thiết lập re-rank
training/distill_collect_grouped.py	Thu thập dữ liệu theo nhóm
training/distill_train_ctx.py	Huấn luyện + xuất
training/validate_student.py	Kiểm tra agreement trên tập giữ lại
training/selfplay.py	A/B self-play
<i>Hướng B — NeRL (bộ lập kế hoạch dự báo)</i>	
agents/NeRL.py	Bộ lập kế hoạch NeRL, một file

hoạch dựa trên mô hình dự báo của nhóm, không có tham số học nhưng đạt 1141.8 trên bảng xếp hạng—cao hơn *ctx2*—và thắng cả *ctx2* lẫn heuristic trong đối đầu, vì nó tối ưu *kết quả* mô phỏng thay vì bắt chước một sở thích cố định. Bài học chung là sự đánh đổi quen thuộc giữa học máy và lập kế hoạch: khi có sẵn một mô hình chính xác, lập kế hoạch dựa trên nó có thể thắng việc học để bắt chước, và hướng hứa hẹn nhất là để học máy đề xuất ứng viên và để lập kế hoạch đưa ra lựa chọn cuối.

Lời cảm ơn

Cảm ơn giảng viên và ban tổ chức Orbit Wars đã cung cấp môi trường thi đấu, benchmark và hướng dẫn, cùng cộng đồng thi đấu vì phản hồi thực tế về hiệu năng agent.

Tài liệu

- [1] M. Zaheer *et al.*, “Deep sets,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” *AISTATS*, pp. 627–635, 2011.
- [3] A. Hussein *et al.*, “Imitation learning: A survey of learning methods,” *ACM Computing Surveys*, vol. 50, no. 2, pp. 1–35, 2017.
- [4] C. E. Garcia, D. M. Prett, and M. Morari, “Model predictive control: Theory and practice—a survey,” *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.